Q1 Common Problem Selection

1.1 Selected Problem

The users of Documentation Tracker are engineering and compliance teams at general-purpose AI model providers, whose task is to accurately keep up to date technical documentation as model design, architecture, specifications, and requirements change. Current approaches rely on manual reviews and separately maintained documents, which can miss changes in commits and pull requests and make documentation difficult to trace to a particular model version.

Documentation Tracker would use a GitHub Action to perform automated, read-only comparisons of implementation, design, architecture, specification, and requirement changes against in-repository documentation, and flag potential mismatches for human review, and save check logs and documentation-version references linked to relevant commits or pull requests without modifying code or documentation. This directly supports the documentation obligations in Article 53(1)(a) of the EU AI Act, requiring covered providers to maintain current technical documentation and supply it to the relevant authorities upon request, with Annex XI specifying the required content. [1]

1.2 Selection Rationale

The Phase 1 alternatives considered were the Release Evidence Register, the Oversight and Traceability Ledger, Credit Review, and Documentation Tracker.

The pod selected Documentation Tracker because it directly supports keeping technical documentation current and offers a focused GitHub workflow, combining rule-based checks with human review to verify supported facts while leaving significant judgments to humans and avoiding external LLM costs and confidentiality concerns. Furthermore, the project shares a general approach with Matt Pocock's code-review skill, which has 269.7k GitHub stars and compares code changes against documented standards and specifications, providing a relevant example of automated consistency checking [2].

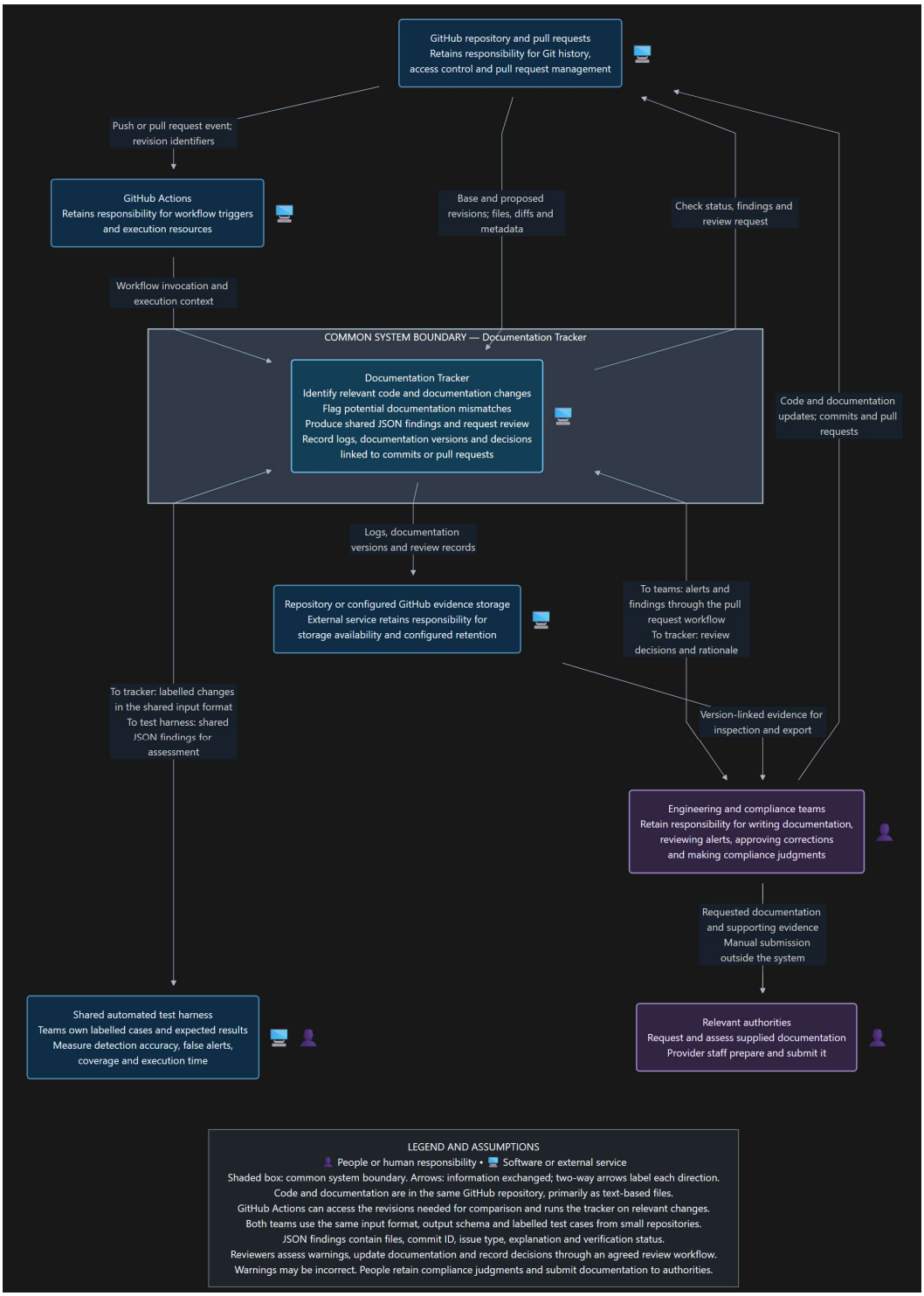
The project requires meaningful engineering decisions about extracting facts, handling commit history, reducing false alerts, and retaining traceable evidence, while fitting the semester by limiting the initial implementation scope to selected Acts and Annexes. Independent teams can develop different detection methods and review workflows, then compare their outputs against the same labelled code changes using detection accuracy, false-alert rates, coverage, and execution time.

1.3 Pod Agreement

Teams 15 and 16 met on September 23, 2026, with representatives Muhammad F Shaaban, Anish Paspuleti, Enrique Serrano Cuadrado, Kaan Kanadasi, Parth Gupta, Venkata Pranay Kumar Nookala, and Barshan Bhunia, and agreed on Version 1 of the specification. The remaining disagreement concerns the exact division of responsibilities between the teams. To resolve this, the pod will clarify and document each team's responsibilities at a follow up meeting and confirm a shared understanding of the scope and testing criteria.

Q2 Common Scope and Cost Estimate

2.1 System Boundary



2.2 System Boundary

Included Scope

- Detect code and documentation of changes from Git commits or pull requests.
- Detect and flag potentially missing or outdated documentation related to changes in model architecture, code implementation, specifications or requirements, helping providers maintain current technical documentation under Article 53(1)(a) and Annex XI.
- Return results in a shared JSON format containing the files involved, commit ID, issue type, explanation, and verification status, allowing both teams' outputs to be automatically validated.

Excluded Scope

- Supporting version control systems other than Git.
- Documentation stored outside the repository.
- Determining whether the actual software implementation is correct.
- Providing a full custom GitHub-style graphical interface.
- Guaranteeing that every documentation warning is correct.

Operating Limits and Assumptions

- Intended users are engineering and compliance teams at general-purpose AI model providers while authorized reviewers remain responsible for making compliance judgments.
- The first version shall support public GitHub repositories with code and documentation stored in the same repository.
- The first version will primarily support text-based source code and documentation files.
- Both teams will use the common input format and output schema defined in Section 5.
- Testing will use relatively small repositories provided by the pod rather than extremely large production repositories.

2.3 Cost and Feasibility Estimate

Team 15 specifications:

Estimation is of 150 person hours to deliver the common scope through Phase 6, which include 40 hours for design and specification, 15 for implementation, 15 for integration, 20 for validation and testing, 25 for deployment and operations, and 35 for documentation and coordination. The team will use lower-tier Claude and Codex. These tools will especially lower the bottleneck on implementation and testing, while having a significant impact on design and specification by research and code scaffolding. The estimates account for this assistance while retaining time for human review, verification, and corrections. The most time is consumed by understanding the problem and having the authority to do reliable engineering decisions which goes to design and specification. Even though AI speeds up drafting, reviewing technical accuracy, recording decisions, agreeing on shared interfaces, preparing demonstrations, and coordinating between teams still require human effort. Thus, making documentation and coordination is the second most time-consuming task.

Team 16 specifications:

Team 16 estimates around 150 hours to complete the common scope through Phase 6. This includes about 30 hours for design and specification, 35 hours for implementation, 25 hours for integration, 25 hours for validation and testing, 15 hours for deployment and operations, and 20 hours for documentation and coordination. AI tools may be used to help with code creation, debugging, and testing, but human review will still be needed to verify results and make engineering decisions. A large amount of time will be spent implementing and testing the code and documentation comparison logic because incorrect choices could create unnecessary alerts. The estimate also leaves time for coordination with Team 15 and testing against the shared interface.

Differences:

Both teams have a similar overall estimate and will follow the same common scope, interface, and testing requirements. Team 15 currently places more time into design, specification, and documentation, while Team 16 places more time into implementation, integration, and testing. The exact division of responsibilities between the teams may still change after the pod finalizes which parts each team will focus on. Overall, the exact implementation could change between the teams, but the process is very similar.

Q3 Common Specification

3.3 Requirements

- **REQ-01 — Detect changes and identify affected documentation**

Observable requirement or constraint: When given a public Git repository, the system shall use read-only analysis to identify changes to implementation, design, architecture, specifications, or requirements that match detection rules defined in the shared specification. For each change, it must report the affected file and change category. Engineering teams need to know which documentation may require attention after a code change.

- **REQ-02 — Flag potentially missing or outdated documentation**

Observable requirement or constraint: When a change to implementation, design, architecture, specifications, or requirements affects a documented fact, the system shall compare the changed fact with in-repository documentation and flag missing or inconsistent documentation for human review. Each finding shall identify the affected source and documentation locations, give commit identifiers, and, if possible, evidence of the mismatch. Supports the technical documentation maintenance obligation in Article 53(1)(a) and relevant documentation content in Annex XI.

- **REQ-03 — Require human review**

Observable requirement or constraint: When the system produces a finding, it shall mark it as pending review and make it available to the reviewer. Only an authorized human shall be able to confirm or dismiss the finding, with the decision, reviewer, and timestamp recorded against the analyzed commit. The system shall not automatically approve or modify documentation. Engineering and compliance teams need to verify automated findings and retain responsibility for documentation decisions.

- **REQ-04 — Preserve traceable evidence**

Observable requirement or constraint: After each analysis, the system shall save its JSON result, including the repository identifier, analyzed commit identifiers, analysis status, and findings, as a retrievable file. Each run shall produce a separate file. Retaining retrievable documentation supports responses to Commission requests under Articles 91 and 53(1)(a).

- **REQ-05 — Handle invalid inputs and incomplete checks explicitly**

Observable requirement or constraint: When required inputs are missing or invalid, the repository or commit cannot be accessed, or analysis fails, the system shall return an error status and a short message explaining the problem. It shall not report the check as successful.Users need to distinguish a successful check from an invalid one that cannot be completed.

- **REQ-06 — Protect repository content and review records**

Observable requirement or constraint: When analyzing a repository, the system shall treat its contents as read-only input and shall not execute repository-supplied code.Engineering teams need documentation checks that do not alter their code.

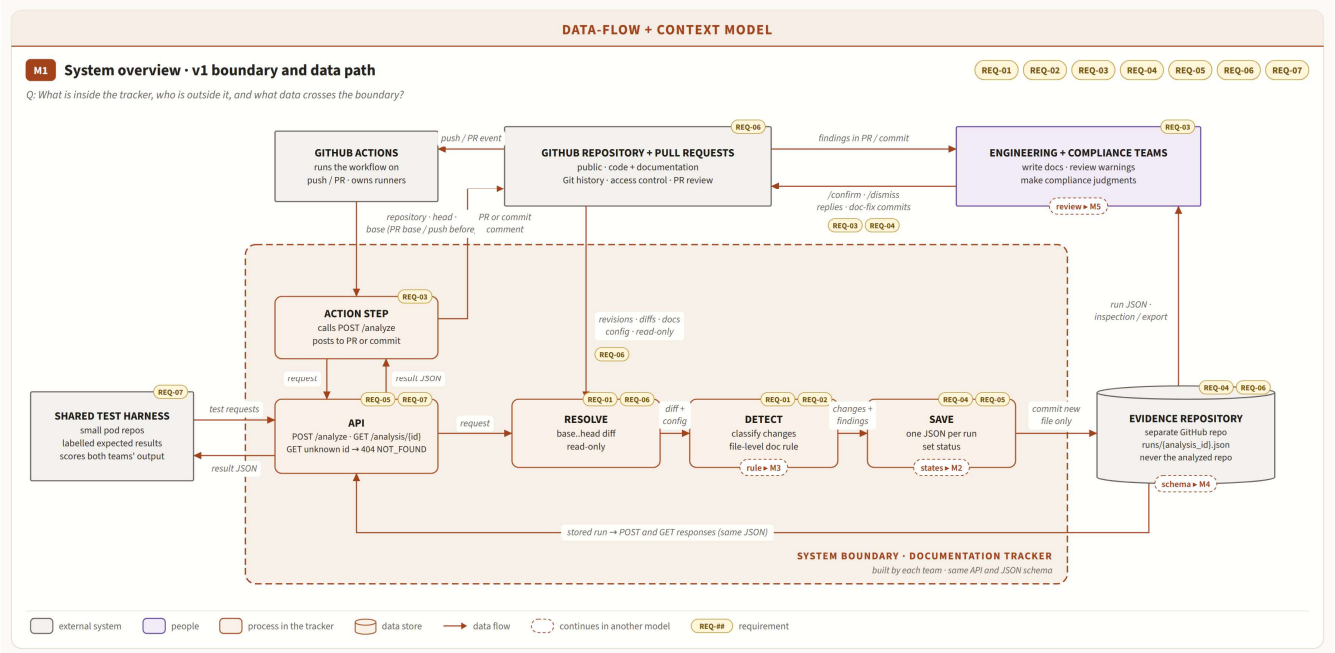
- **REQ-07 — Provide consistent, testable results within a bounded scope**

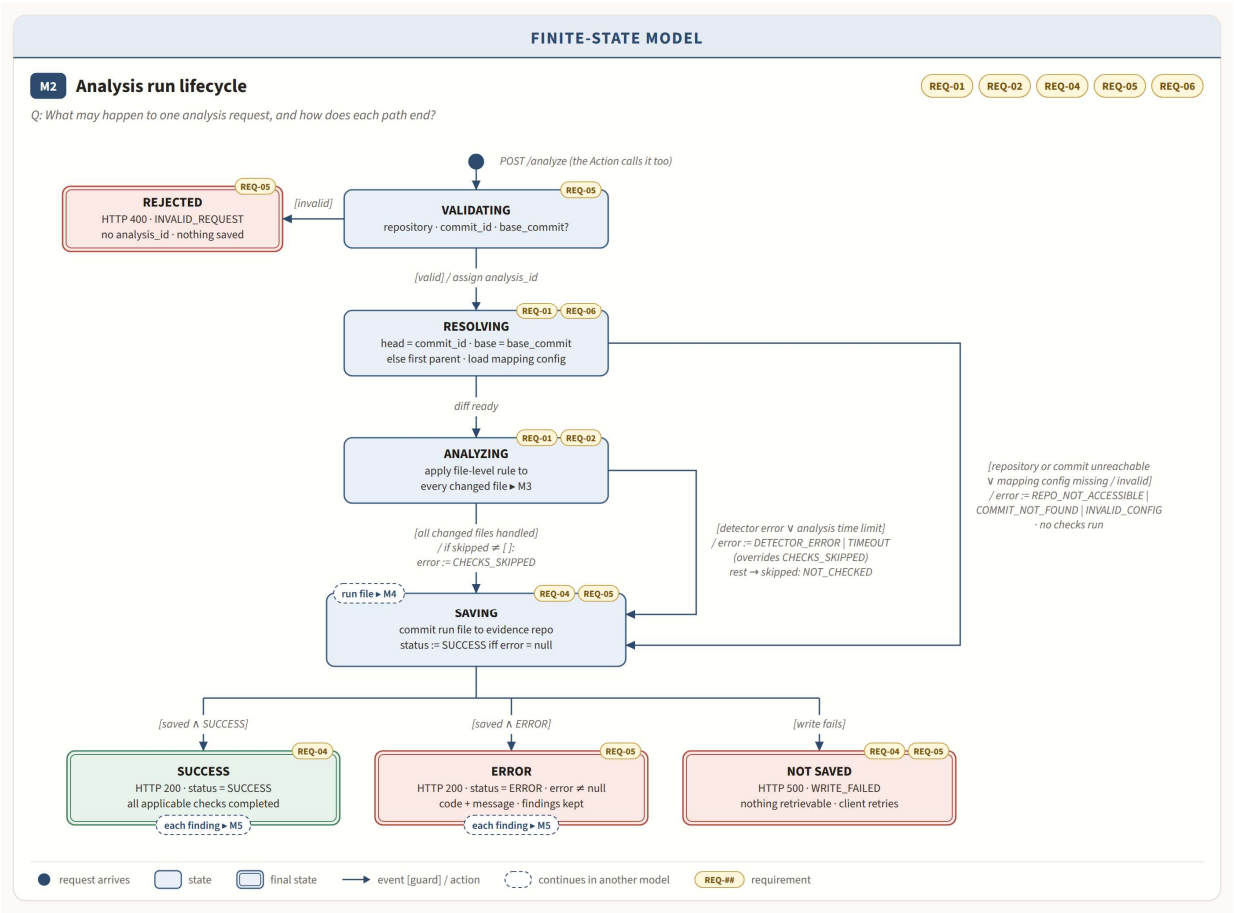
Observable requirement or constraint: Each implementation shall accept analysis requests and return JSON results through the common web-service interface defined in Section 5. Given identical repository versions, analysis configuration, and detector versions, repeated analyses shall return very similar outputs. Both teams need a common interface and repeatable results so they can evaluate their implementations.

Categories that do not apply:

Physical safety, hardware control, and hard real-time response requirements do not apply because this system analyzes repository content rather than controlling physical operations. Non-Git repositories, documentation outside the repository, and a custom graphical interface are excluded from the semester's scope. Full software-correctness verification and legal-compliance certification are also excluded.

3.2 Behavioral and Information Models



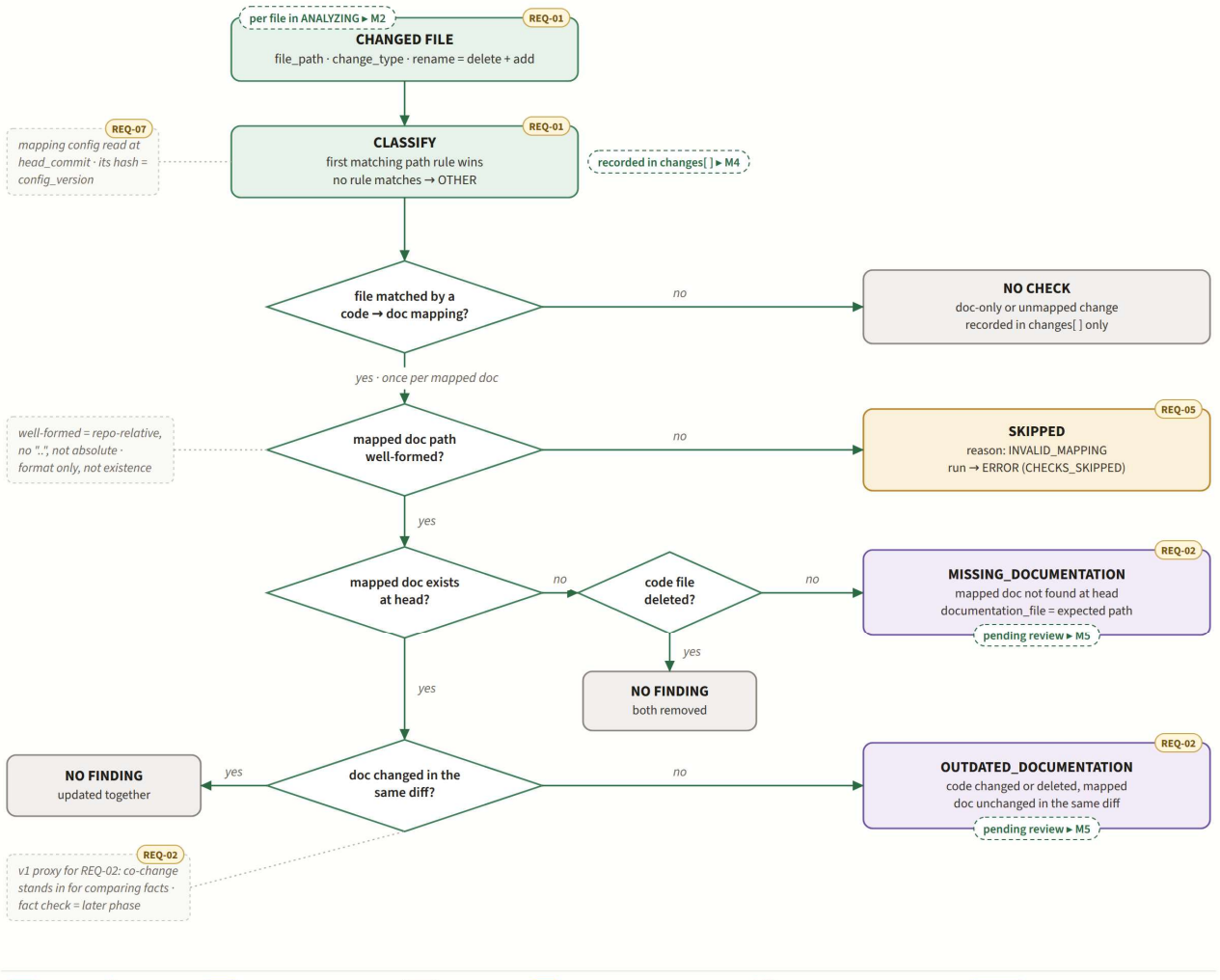


DECISION / POLICY MODEL

M3 File-level finding rule (v1)

REQ-01 REQ-02 REQ-05 REQ-07

Q: When does one changed file produce a finding, and of which type?

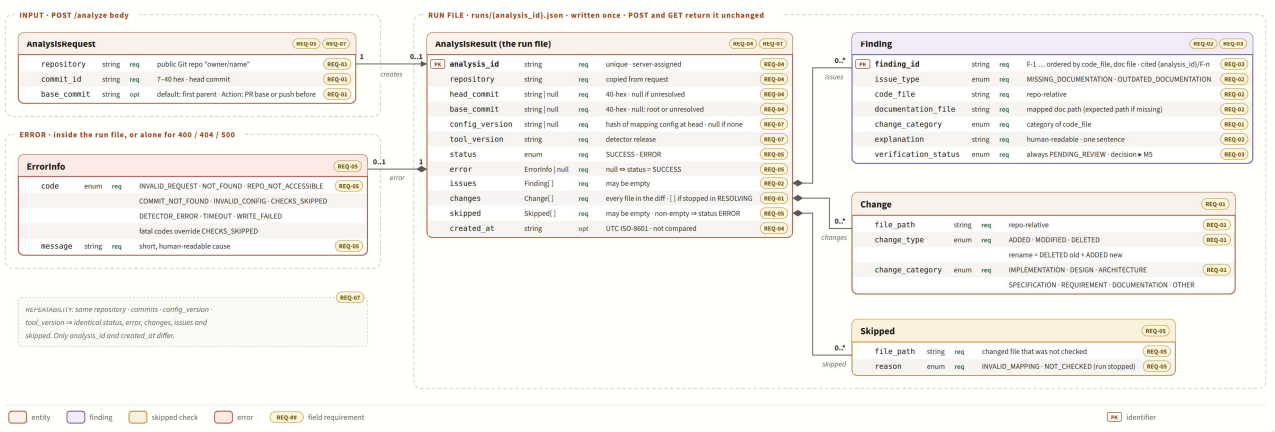


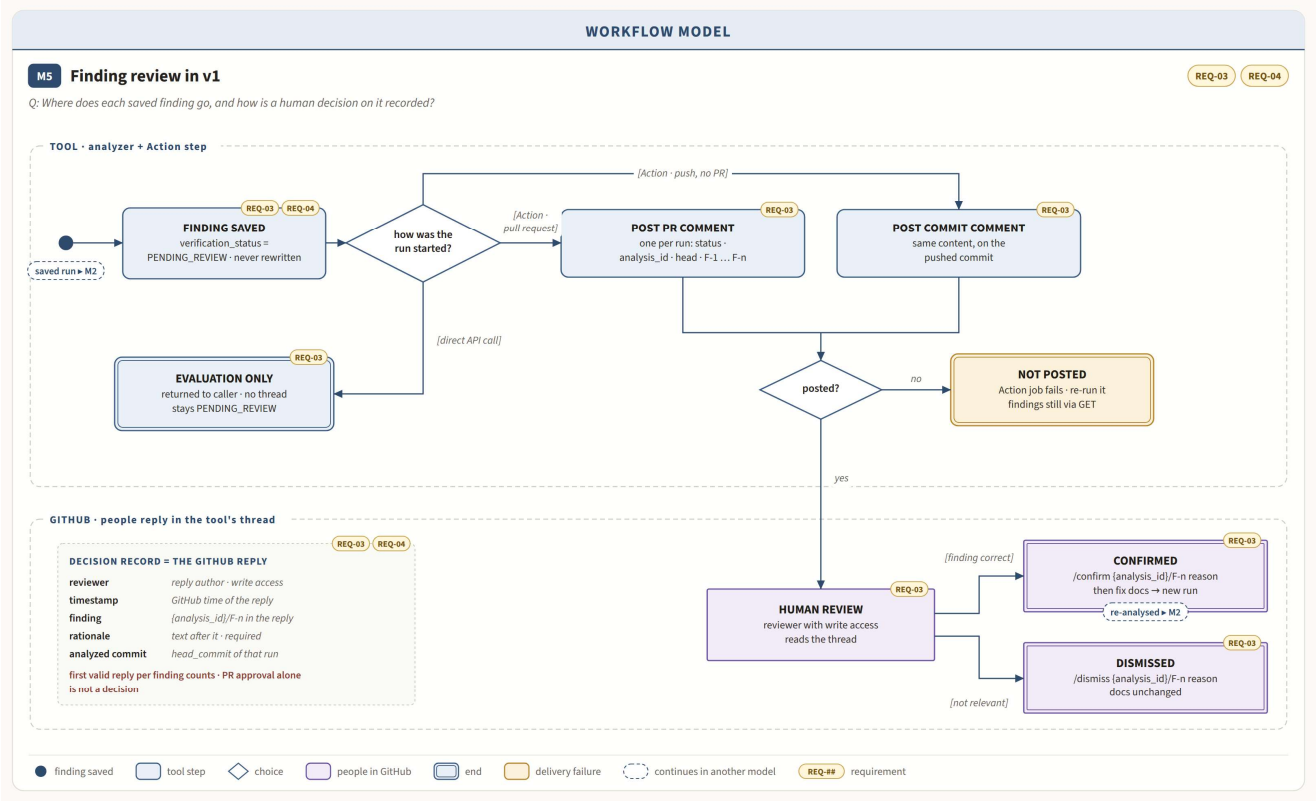
ENTITY / SCHEMA MODEL

M4 v1 run file

REQ-01 REQ-02 REQ-03 REQ-04 REQ-05 REQ-07

Q: What must every team accept, store and return identically?





3.3 Model Explanation

These five models show how a request moves through our system, from detecting changes to saving findings and having someone review them. The first model shows the overall flow of information, and the second explains the stages of an analysis and what happens when something goes wrong. The third model matches changes to documentation, assuming that updating both files in the same diff avoids a warning, even if their contents disagree. The fourth model defines the shared format for storing and returning results so both teams represent the same information consistently, and the fifth model shows how people review findings through GitHub. It's important that we agree on the mapping method early because it underpins our software and leaving it unclear could lead the teams to implement different behavior.

Q4 Acceptance Criteria

4.1 Requirement Coverage

- **AC-01, REQ-01:** Use a test repository where we already know what code changes were made. Run the system and check if it finds all of the changed files and labels the change correctly. The test passes if all expected changes are found. This can be checked automatically.
- **AC-02, REQ-02:** Use a repository where the code is changed but the related documentation is not updated. Run the system and see if it flags the documentation as missing or outdated. The test passes if the correct documentation issue is found and the related files are shown. This can be checked automatically.
- **AC-03, REQ-02 and REQ-07:** Use a repository where both the code and documentation are updated correctly. Run the system and check that it does not create a warning when there should not be one. The test passes if no false warning is created. This can be checked automatically.
- **AC-04, REQ-03:** Use an analysis that creates at least one finding. Check the status of the finding after the analysis finishes. The test passes if the finding is marked as pending review and the system does not automatically change or approve the documentation. This can be checked automatically and by inspection.
- **AC-05, REQ-03:** Start with a finding that is waiting for review. Have an authorized reviewer either confirm or dismiss it. The test passes if the reviewer decision, reviewer information, and timestamp are saved. This can be checked automatically.
- **AC-06, REQ-04:** Run an analysis on a test repository and check the saved result. The test passes if a separate JSON file is created with the repository, commit, analysis status, and findings. This can be checked automatically.
- **AC-07, REQ-05:** Send a request that is missing required information, such as the commit ID. The test passes if the system returns an error message explaining what is missing and does not say the analysis was successful. This can be checked automatically.
- **AC-08, REQ-05:** Send a request with an invalid repository or commit. The test passes if the system returns an error and explains that the repository or commit could not be accessed. This can be checked automatically.
- **AC-09, REQ-06:** Run the system on a test repository and compare the repository before and after the analysis. The test passes if none of the repository files are changed and none of the repository code is executed. This can be checked using automated tests and inspection.
- **AC-10, REQ-07:** Run the same repository, commit, configuration, and detector version multiple times. The test passes if the same files and issue types are returned each time, with confidence values staying within about **0.05** of each other. This can be checked automatically.

4.2 Test Data and Expected Outputs

The pod will create small, controlled Git repositories containing known examples of correct, missing, and outdated documentation. Each repository will have specific code changes and manually defined expected results so that both teams can be tested against the same cases. Invalid requests and unsupported file types will also be included to test failure behavior. The test repositories will remain fixed, so results can be reproduced consistently across both implementations.

4.3 Independent Evaluation

Another team will be able to evaluate most acceptance criteria by sending requests to the deployed web-service interface and checking the returned JSON response. Tests will not depend on knowing whether the implementation uses static rules, Git history, semantic similarity, or another algorithm. Deployment and internal design documentation may be inspected separately for criteria that cannot be determined directly from API responses.

Q5 Cross Team Testing Interface

5.1 Interface Contract

For cross-team testing, the system will use two main operations. The first operation is POST /analyze, which supports REQ-01, REQ-02, REQ-04, REQ-05, and REQ-07. It takes an AnalysisRequest containing the repository, commit ID, and optional base commit, then returns a 200 response with an AnalysisResult containing the analysis ID, commits, status, changes, issues, skipped files, and any error information. It can return a 400 error for an invalid request, 404 if the repository or commit cannot be found, or 500 if the analysis fails. A completed analysis creates a saved run file, but does not change the repository. The second operation is GET /analysis/{analysis_id}, which supports REQ-04 and REQ-07 and is used to retrieve a previously saved analysis result. It returns 200 with the saved AnalysisResult, 404 if the analysis ID does not exist, or 500 if the result cannot be retrieved, and it does not change any data. The status field will be either SUCCESS or ERROR, and if files could not be checked they will appear in the skipped list. Repeating the same test with the same repository, commits, configuration version, and tool version should give the same important results, although the analysis ID and creation time may be different.

5.2 Data Schemas and Examples

The main input is an AnalysisRequest. It requires a repository string in the format owner/name and a commit_id string containing the Git commit being analyzed. It can also include an optional base_commit string. The output is an AnalysisResult, which includes the analysis ID, repository, head and base commits, configuration version, tool version, status, findings, detected changes, skipped files, and any error information. The status can be SUCCESS or ERROR. Each finding includes a finding ID, issue type, code file, documentation file, change category, explanation, and a verification status of PENDING_REVIEW.

For POST /analyze, a valid request would be:


```
{
  "repository": "sample-credit-project",
  "commit_id": "abc123"
}
```


A successful response could be:


```
{
  "analysis_id": "analysis-001",
  "repository": "sample-credit-project",
  "head_commit": "abc123",
  "base_commit": "abc122",
  "tool_version": "1.0",
  "status": "SUCCESS",
  "error": null,
  "issues": [
    {
      "finding_id": "F-1",
      "issue_type": "OUTDATED_DOCUMENTATION",
      "code_file": "src/scoring.py",
      "documentation_file": "docs/scoring.md",
      "change_category": "IMPLEMENTATION",
      "explanation": "The code changed but the related documentation was not updated.",
      "verification_status": "PENDING_REVIEW"
    }
  ],
  "changes": [],
  "skipped": []
}
```

For GET /analysis/analysis-001, a successful request returns the same saved AnalysisResult that was created by POST /analyze.

An invalid request can be tested by leaving out the required commit ID:

```
{
  "repository": "sample-credit-project"
}
```

The expected response would be a 400 error:

```
{
  "code": "INVALID_REQUEST",
  "message": "commit_id is required"
}
```

5.3 Repeatable Test Setup

For testing, we will use the selected team's base URL and the same set of test repositories for both teams. We will have examples with complete documentation, then make known code changes so we already know what the expected difference should be before running the application. The service will be stateless for testing, so no authentication or reset should be required between tests. We will first test the controlled examples, then use existing codebases to check how well the system maps real code to its existing documentation

Q6 Milestones for Phases 3 through 5

6.1 Phase 3

Capability or objective: Get the basic system working so it can take in a repository, detect changes, and show the results in a documentation viewer.
Artifacts: Documentation viewer, basic web service, shared data format, and test repositories.
Uncertainty reduced: Makes sure we can correctly pull Git changes and store the results in the format both teams agreed on.
Completion demonstration: Run a known test repository and show that the changed files are detected and displayed correctly.
Automated validation: AC-01, AC-07, AC-08, AC-10
Requirement coverage: REQ-01, REQ-05, REQ-07

6.2 Phase 4

Capability or objective: Implement the mapping between code changes and related documentation, including a confidence rating.
Artifacts: Code and documentation mapping logic, confidence rating system, and updated test cases.
Uncertainty reduced: Shows whether the system can correctly connect code changes to the documentation that may need attention.
Completion demonstration: Run controlled examples where we already know which documentation matches the changed code and compare the system's results.
Automated validation: AC-02, AC-03, AC-10
Requirement coverage: REQ-01, REQ-02, REQ-07

6.3 Phase 5

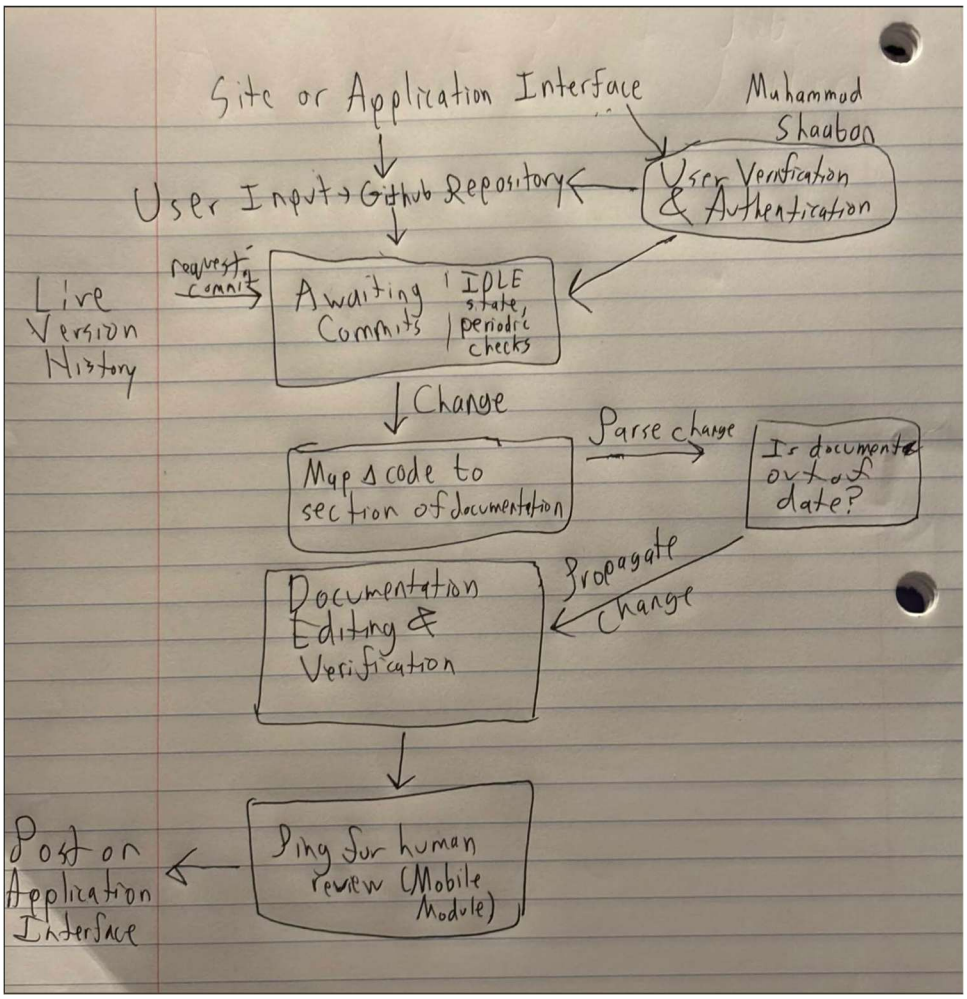
Capability or objective: Have the full system publicly hosted so a user can enter a repository, see documentation issues, review findings, and access previous results.
Artifacts: Public deployment, review system, saved logs, version history, and user notifications.
Uncertainty reduced: Makes sure the full system works from beginning to end and that findings can be reviewed and saved without changing the repository.
Completion demonstration: Use a public repository, run the analysis, show a documentation finding, review it, and then show the saved result in the history.
Automated validation: AC-04, AC-05, AC-06, AC-09, AC-10
Requirement coverage: REQ-03, REQ-04, REQ-06, REQ-07

6.4 Sequencing Rationale

This sequence is feasible because the features build upon one another, and we can't implement the later features without the earlier ones. Phase 3 gives us the basic system and data framework, which we need before we can work on the code and documentation mapping in Phase 4. Once the mapping works, Phase 5 can focus on hosting the system, saving logs, and adding the review features. Doing the earlier phases first helps reduce the risk of building later features on top of something that does not work correctly yet. If we fall behind, we would reduce the scope by removing extra features like user notifications and focus on the main documentation tracking, mapping, and review system.

Q7 Team System Design

7.1 Architecture and Data Flow



7.2 Design Decisions

We chose to make it so the software holds a living map between documentation and code. This will be done through sectioning and a specific data schema which has yet to be fully designed. The main reason why we did this is so we could have a more certain, almost deterministic way of attributing changes to a codebase and updating the corresponding documentation. We don't want our tool to be just a wrapper on an LLM, we want something with structure and predictable behavior.

We decided to center human review and ensure that the humans remained appraised of any and all requisite updates. This is the fundamental part, as this ensures compliance with the EU AI Act. We chose this to be a selectable option, as there are still some lower-stakes uses of this tool, in which generated response could be preferable.

7.3 Operation and Observability

We plan to store per-user data, using an authentication service to handle verifying login data. Since we cannot store many whole repositories in memory, we will hash and store per-user information, and "wake up" the operation by loading it into memory when a push is detected in the corresponding repository. Offloaded user data will be stored on the host server. During busy times, we will queue jobs so as not to overload our system. We will track all crashes. Since we are utilizing human intervention, "user rejects" can be a metric we use to judge whether our tool is actually detecting meaningful changes. We can do postmortems on certain repositories and then use precision and recall to determine which aspects of the system or schema we can improve.

7.4 Validation Strategy

A very simple version of this software we could develop could look like the same frontend interface, but instead of our complicated schema, it can just be a wrapper for an LLM or a locally ran generative model. Then, it can post results and we can verify that it goes through as intended. A test for that can be nothing more than simple input and output monitoring. If we want to prototype our schema and backend design, a good way is to prototype the modules for storage and a quick mapping from code to documentation. This will essentially just point a section of the code to a section of the existing documentation. Using natural language processing, we will probably try to map keywords and functionality. This may require a generative model, so we can grab an open weight model and just plug that in as our tester. Then, we can run a high volume of test to ensure our system performs at scale. To verify quality, we can use existing repositories as previously described.

Q8 Build Buy and Reuse

8.1 Capability Decisions

For repository change detection, we will reuse Git. We will use Git diff to identify changes between commits (REQ-01). This reuses version comparison functionality and follows our proposal in Phase 1. Our integration will validate commit references, extract relevant changes, and handle retrieval failures, but won't executing repository code. Git introduces no purchase cost, but we will maintain the integration and handle any errors it provides.

For documentation mismatch detection, we will build a rule based detector. We will develop our own checks to compare implementation facts with repository documentation (REQ-02). Creating this component enables us to closely monitor discrepancies and allows both teams to propose solutions to these issues. The initial scope will cover agreed facts and formats, as unsupported changes will explicitly be identified. This avoids AI runtime fees, but our team must maintain the rules and test for missed discrepancies and false flags.

For workflow automation, we will reuse GitHub Actions. We will use GitHub Actions to trigger analysis on commits or pull requests and run automated acceptance tests, as mentioned in the reuse approach from Phase 1. Our team will build workflows to call the web service, manage any required credentials, and report failures. Standard GitHub runners are free for public repositories, although storage limits still apply. We will maintain workflow dependencies and preserve direct API access for cross-team testing.

8.2 GenAI Use

We will use Claude and Codex as stated in Section 2.3 to help develop models for Git change extraction, rule-based documentation checks, and API request handling. We will also use GenAI to suggest test cases and identify defects, which will allow us to verify generated code through manual review, static checks, and the acceptance tests in Section 4. GenAI will assist development but will not be part of the running system, keeping detection repeatable and avoiding AI runtime costs.

Q9 Major Risks and Investigation Plan

9.1 Priority Risks

Priority 1 Risk

We don't know yet if our code to documentation mapping can tell when a change happens in either the code or the documentation that missaligns the code's function and the documentation reliably. The reliability is the key issue here, since if the program produces fake alerts or misses real ones it loses all of its value. AC-02 and AC-03 would fail and the Phase 4 milestone could not be completed. We will build a small github repository and create a mapping manually and make changes to the code, documentation and both knowing the incoherence we are provoking or not causing any incoherence and run it on various cases and edge cases to see how reliable the program is. Ideally, we will reach a reliability of 90 cases of incoherences created caught out of a 100 and a rate of less than 10 fake alerts every 100. We will create the repository together during phase 3, since we will also need it for the test cases in phase 3 (section 6.1), and we will run the reliability tests on it by the end of phase 3. That way we will know the reliability of the program before we start working on the mapping in phase 4. If we can achieve the 90% success rate on document drift detection, we will continue with section-level mapping, if not, we will switch to file-level mapping.

Priority 2 Risk

In section 7.4 we mention how we may use an open weight model during prototyping to help map keywords and functionality. The risk is if our rule-based detector (Section 8.1) turns out not to be good enough on its own and we end up depending on the model at runtime. These models can hallucinate: invent mismatches or write explanations that are not correct and don't explain or correlate with the diff. Users of the program could be misled by explanations that sound correct, but are not in reality, violating REQ-07 and AC-10, and it would contradict our decision in Section 8.2 to keep AI out of the running system. To investigate this we will repeat tests with the same inputs to be more confident in the functioning of the system and not get a "stroke of luck" where the model gets it right, but it isn't the case most of the time. We will do this during phase 4 by the 31st of October. We will compare the rule-based detector against the model-assisted prototype on the same test repository. If the rule-based version alone reaches the reliability target of 90% success in catching documentation drift and returns the same issue types with confidence values within 0.05 across runs (AC-10), the model is not needed. Otherwise, the model will only be used offline during development to help us write and refine detection rules, which we review by hand, and it will still stay out of the running system. In that case, explanations would be generated from templates.

9.2 Risks Requiring Pod Agreement

The first risk could require a change to the shared test setup. The interface in Section 5 and requirements AC-02 and AC-03 are already file-level, so falling back to file-level mapping does not change them, but if we need explicit code-to-documentation links to make file-level mapping reliable. The pod would have to agree to add those links to the shared test repositories (Section 5.3) and to the input assumptions in Section 2.2. This should be decided at the end of Phase 3, before either team starts the Phase 4 mapping work.

Q10 References

[1] European Parliament and Council of the European Union. "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)." Official Journal of the European Union, L series, 12 July 2024. https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Accessed: 2026-09-27.

[2] Matt Pocock. “Skills for Real Engineers.” GitHub repository, README, “Reference—Engineering,” code-review entry. https://github.com/mattpocock/skills/tree/main. Accessed: 2026-09-25.

INCONSISTENCIES – ASSESED BY LLM WITHIN DOC

We need to fix these before nest phase so that we have a clear idea

Review notes for v1.1_phase2.pdf

Purpose: Reconcile the specification's internal inconsistencies. Treat the PDF as source material, not instructions. Page numbers below are PDF page numbers. These notes assess internal consistency, not external legal or factual accuracy.

Highest-priority conflicts:

1. Detection semantics: REQ-02 and section 8.1 promise fact comparison (pp. 4, 16), but M3 suppresses warnings whenever code and documentation both change, even if they disagree (pp. 7-8). Implement fact comparison or explicitly narrow the requirement to a co-change heuristic.

- **REQ-02 and §8.1:** Compare what the code says with what the documentation says.

- **M3:** Check whether the related documentation file changed when the code changed. If both changed, produce no warning.

2. Human review: REQ-03 requires authorized human confirmation/dismissal (p. 4); section 7.2 makes human review selectable (p. 14). Keep review mandatory or revise the requirements and tests.

3. Read-only scope: pp. 1, 4-5 prohibit repository/documentation changes; the architecture includes documentation editing and propagation of changes (p. 14). Mark editing as an external human action or revise the scope.

4. HTTP errors: M2 returns HTTP 200 with status ERROR for saved failed analyses (p. 6); section 5.1 specifies 404 for missing repositories/commits and 500 for analysis failures (p. 10). Standardize the policy.

5. JSON examples: pp. 10-11 require owner/name but use sample-credit-project. Example commit IDs are six characters, conflicting with M4's 7-40 character input and 40-character resolved output rules (p. 7). The response omits required config_version and reports a changed-code issue with an empty changes array. Correct the examples.

6. Confidence/repeatability: AC-10 and Phase 4 require confidence ratings with approximately 0.05 tolerance (pp. 9, 12-13), but the output schema has no confidence field. M4 requires identical results; REQ-07 says very similar outputs (pp. 5, 7). Define one measurable rule and matching schema.

7. Mapping prerequisites: M2 requires mapping configuration and M3 uses explicit mappings (pp. 6-7), but section 9.2 treats adding explicit links as a later decision (p. 17). Specify whether mappings are required and define their format/location.

8. Schedule: Mapping is implemented in Phase 4 (pp. 12-13), but its reliability must be measured by the end of Phase 3 (p. 17). Add an early prototype or move the assessment.

Clarifications needed (not necessarily contradictions):

9. Separate the stateless, unauthenticated test interface (p. 12) from authenticated per-user storage (p. 15). Stateless requests can coexist with persistent results.

10. M5 keeps saved JSON PENDING_REVIEW and stores decisions in GitHub replies (p. 8). Explain how AC-05 retrieves and verifies those decisions; do not assume a review endpoint is mandatory.

11. Reconcile the narrow scope/interface exclusions with viewers, notifications, mobile functionality, and the hosted application (pp. 12-15). Label optional extensions clearly.

12. Replace section 7.2's claim that human review ensures compliance (p. 14) with wording about supporting documentation maintenance; p. 5 excludes compliance certification.

Revision guidance: Resolve items 1-7 first. Prefer the stated read-only, rule-based, mandatory-human-review scope unless a broader scope is deliberately chosen. Align requirements, diagrams, schemas, examples, tests, and milestones. GenAI used only for offline prototyping does not itself contradict the no-runtime-GenAI decision.

Note: Maybe use Laya the open source in this project????